



Unidad IV. Desarrollo de Aplicaciones web dinámicas	ACTIVIDAD: DSW-A11. Aplicación web MVC con SlimPHP y PHP – Centro de Cómputo FEI
TIPO: Actividad individual	

OBJETIVO:

El estudiante desarrollará una aplicación web dinámica utilizando PHP, el framework SlimPHP y el patrón de diseño Modelo–Vista–Controlador (MVC), a partir de la página del Portal Institucional del Centro de Cómputo FEI trabajada en actividades anteriores, con la finalidad de dar funcionamiento real al formulario de reporte de incidente, almacenar la información en una base de datos MySQL y consultar los registros capturados mediante una vista adicional de la aplicación.

DESCRIPCIÓN:

En esta actividad, el estudiante retomará la versión más reciente del Portal Institucional del Centro de Cómputo FEI, desarrollada en la práctica DSW-A10. Página HTML, CSS, JavaScript, AJAX – Centro de Cómputo FEI, en la cual ya se integraron funcionalidades de frontend como interactividad con JavaScript, validación de formulario, carga dinámica de datos con fetch() y visualización de información en formato JSON

A partir de esa base, ahora se incorporará la parte backend utilizando PHP y SlimPHP, organizando el proyecto con el patrón MVC, tal como se muestra en la práctica de referencia sobre MVC con SlimPHP. En esta nueva versión, el formulario dejará de ser una simulación del lado del cliente y pasará a registrar incidentes reales en una base de datos MySQL. Además, el usuario podrá adjuntar un archivo de evidencia, el cual será almacenado en el servidor, y habrá una nueva vista para consultar los incidentes ya capturados.

La finalidad es que el estudiante comprenda la relación entre:

- Modelo, que representa los datos;
- Vista, que presenta la información al usuario;
- Controlador, que procesa las solicitudes y coordina la lógica;
- Capa de acceso a datos, que interactúa con MySQL;
- y rutas, que conectan las URL con los controladores.

De esta manera, el alumno reconocerá cómo una aplicación web moderna integra estructura, presentación, comportamiento y lógica del servidor dentro de una arquitectura organizada y mantenible.

Estructura del proyecto y relación con MVC

El proyecto contendrá las siguientes partes:



```
centrocomputo_mvc/  
├── app/  
│   ├── env.php  
│   ├── routes.php  
│   └── settings.php  
├── public/  
│   ├── css/  
│   │   └── estilos.css  
│   ├── img/  
│   │   └── ...  
│   ├── uploads/  
│   └── index.php  
├── src/  
│   ├── Controllers/  
│   │   ├── HomeController.php  
│   │   └── IncidenteController.php  
│   ├── Data/  
│   │   └── DataContext.php  
│   ├── Models/  
│   │   └── Incidente.php  
│   ├── Settings/  
│   │   └── Settings.php  
│   └── Views/  
│       ├── Home/  
│       │   ├── Index.php  
│       │   └── Incidente/  
│       │       └── Lista.php  
├── vendor/  
│   └── ...  
├── bd.sql  
├── composer.json  
└── composer.lock
```

1. app/

Esta carpeta contiene la **configuración general** de la aplicación.

env.php

Guarda los datos del entorno, por ejemplo:

- host de la base de datos,
- nombre de la base,
- usuario,
- contraseña.

settings.php

Carga la configuración general del sistema y combina los valores del entorno.



routes.php

Define las **rutas** de la aplicación, por ejemplo:

- / → mostrar portal
- /incidente/guardar → guardar reporte
- /incidentes → consultar reportes

2. public/

Contiene los **archivos públicos** que el navegador sí puede cargar directamente.

public/css/

Aquí va la hoja de estilos.

public/img/

Aquí van las imágenes del portal.

public/uploads/

Aquí se almacenan los archivos de evidencia que sube el usuario desde el formulario.

public/index.php

Es el punto de entrada principal de la aplicación.

Cuando el usuario entra a la aplicación desde el navegador, este archivo es el primero que se ejecuta.

Aquí se:

- Carga Composer
- Inicia slimphp
- Registra servicios
- Carga las rutas
- Ejecuta la aplicación

3. src/

En esta carpeta se encuentra el **código fuente principal** de la aplicación y donde se gestiona el patrón de arquitectura MVC.

- A. **src/Models/** → Modelo (representa los datos de la aplicación)

```
src/  
└─ Models/  
    └─ Incidente.php
```

- **Incidente.php** representa un incidente reportado por un usuario. Contiene propiedades como:
 - nombre
 - correo



- rol
- tipo
- descripción
- evidencia
- fecha de registro

B. **src/Views/** → Vista (es la parte que se muestra al usuario)

```
src/  
└─ Views/  
    └─ Home/  
        └─ Index.php  
    └─ Incidente/  
        └─ Lista.php
```

Home/ Index.php

Muestra el portal principal del Centro de Cómputo y el formulario de reporte.

Incidente/Lista.php

Muestra la lista de incidentes registrados.

C. **src/Controllers/** → Controlador (recibe la solicitud del navegador, decide qué hacer y coordina el flujo)

```
src/  
└─ Controllers/  
    └─ HomeController.php  
    └─ IncidentController.php
```

HomeController.php

Se encarga de mostrar la vista principal del portal.

IncidenteController.php

Se encarga de:

- Recibir los datos del formulario
- Procesar la subida del archivo
- Llamar a la capa de datos
- Guardar el incidente
- Mostrar la vista con los registros

Otras partes importantes que apoyan MVC

D. **src/Data/** (capa de acceso a datos)

```
src/  
└─ Data/  
    └─ DataContext.php
```



DataContext.php Se conecta a MySQL y realiza operaciones como:

- Insertar incidentes
- Consultar incidentes

E. src/Settings/ (clase permite acceder a la configuración del sistema desde otras partes del proyecto)

```
src/  
└─ Settings/  
   └─ Settings.php
```

F. vendor/ (la genera automáticamente Composer, y contiene SlimPHP, PSR-7 y otras dependencias)

```
vendor/  
└─ ...
```

G. bd.sql

Es el script para crear la base de datos y la tabla incidente.

Sirve para:

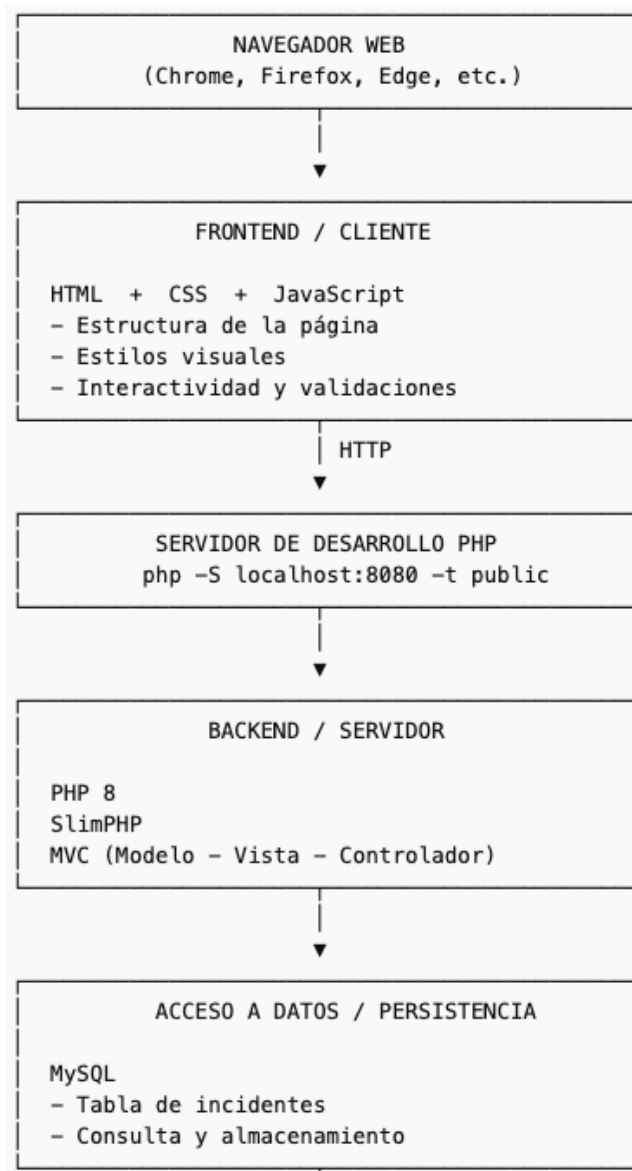
- Crear la estructura inicial
- Dejar lista la base para trabajar con la aplicación

H. composer.json

Define:

- Las dependencias del proyecto
- El autoload
- La configuración de Composer

Stack de desarrollo



MATERIAL Y EQUIPO REQUERIDOS:

- Visual Studio Code o cualquier otro editor de código.
- Navegador web actualizado (Chrome o Firefox).
- Carpeta del proyecto desarrollada en la actividad DSW-A10.
- Intérprete de PHP 8 instalado en el equipo.
- Composer instalado para administrar dependencias de PHP.
- MySQL instalado y funcionando localmente.
- Acceso a terminal o consola.
- Carpeta de trabajo local para crear el nuevo proyecto MVC.
- Permisos de escritura en la carpeta del proyecto para almacenar archivos subidos por el usuario.



Revisa la práctica DSW-A11a-requisitosPHP_MySQL_Composer, para instalar los requisitos necesarios para esta práctica.

PROCEDIMIENTO:

Paso 1 – Crear una copia del proyecto base

Realiza una copia de la carpeta del proyecto elaborado en la actividad **DSW-A10**. Esta copia será la base para integrar backend con PHP.

Puedes asignarle un nombre como:
centrocomputo_mvc o DSW-A11

Conserva los recursos del frontend que ya has trabajado:

- css/estilos.css
- carpeta img
- estructura visual del portal
- contenido HTML que servirá como base para las vistas

En esta práctica ya no trabajarás con index.html como página principal estática, sino con una vista PHP renderizada desde SlimPHP que crearás más adelante.

Paso 2 – Instalar SlimPHP y dependencias

Abre la carpeta del proyecto en Visual Studio Code y luego abre la terminal. Ejecuta los siguientes comandos:

```
composer require slim/slim:"4.*"  
composer require slim/psr7:"1.*" --with-all-dependencies  
composer require php-di/php-di
```

Estas dependencias permiten:

- Crear la aplicación con Slim
- Manejar solicitudes y respuestas HTTP
- Definir rutas para la aplicación

Paso 3 – Crear la estructura general del proyecto

Dentro de la carpeta del proyecto crea la siguiente estructura:

```
app  
src  
src/Controllers  
src/Data
```



```
src/Models
src/Settings
src/Views
src/Views/Home
src/Views/Incidente
public
public/css
public/img
public/uploads
```

Ahora copia tus recursos estáticos a la carpeta `public`:

- Copia `estilos.css` a `public/css/`
- Copia tus imágenes a `public/img/`
- Copia tu script a `public/js`
- Copia los datos a `public/datos/`

Función de cada carpeta

- `app`: contiene configuración general y rutas.
- `src/Controllers`: contiene los controladores.
- `src/Models`: contiene los modelos.
- `src/Data`: capa de acceso a datos.
- `src/Settings`: acceso a configuración.
- `src/Views`: vistas PHP.
- `public`: archivos públicos de la aplicación.
- `public/uploads`: almacenamiento de evidencias subidas por el usuario.

Paso 4 – Crear la base de datos

En la raíz del proyecto crea un archivo llamado **bd.sql** con el siguiente contenido:

```
DROP DATABASE IF EXISTS centro_computo;
CREATE DATABASE centro_computo CHARACTER SET utf8mb4;
USE centro_computo;

CREATE TABLE incidente (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL,
    correo VARCHAR(120) NOT NULL,
    rol VARCHAR(50) NOT NULL,
    tipo VARCHAR(80) NOT NULL,
    descripcion TEXT NOT NULL,
    evidencia VARCHAR(255) DEFAULT NULL,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```




Abre el cliente de MySQL y ejecuta:

```
source ruta_completa_de_tu_proyecto/bd.sql
```

Ahora crea el usuario de la aplicación:

```
CREATE USER cc_user@localhost IDENTIFIED BY 'cc2026*';  
GRANT ALL PRIVILEGES ON centro_computo.* TO cc_user@localhost;  
SHOW GRANTS FOR cc_user@localhost;
```

Con ello quedará lista la base de datos

Paso 5 – Crear el archivo de entorno

Dentro de app, crea el archivo env.php:

```
<?php  
  
return function (array $settings) {  
    $settings['db']['host'] = 'localhost';  
    $settings['db']['database'] = 'centro_computo';  
    $settings['db']['username'] = 'cc_user';  
    $settings['db']['password'] = 'cc2026*';  
  
    $settings['env'] = 'dev';  
  
    return $settings;  
};
```

Este archivo almacena los datos de conexión a la base de datos. Es parte de la **configuración de la aplicación**, necesaria para que otras capas funcionen correctamente.

Paso 6 – Crear la configuración general

Dentro de app, crea settings.php:

```
<?php  
  
error_reporting(E_ALL);  
ini_set('display_errors', '1');  
ini_set('display_startup_errors', '1');  
  
date_default_timezone_set('America/Mexico_City');  
  
$settings = [];  
  
$settings = (require __DIR__ . '/env.php')($settings);
```



```
return $settings;
```

Este archivo define la configuración general y carga el entorno.

Paso 7 – Crear la clase Settings

Dentro de `src/Settings`, crea `Settings.php`:

```
<?php

namespace App\Settings;

class Settings
{
    private array $settings;

    public function __construct(array $settings)
    {
        $this->settings = $settings;
    }

    public function get(string $key = '')
    {
        return empty($key) ? $this->settings : $this->settings[$key];
    }
}
```

Esta clase sirve como apoyo para acceder a la configuración desde los controladores y la capa de datos.

Paso 8 – Configurar el autoload en Composer

Abre `composer.json` y agrega:

```
"autoload": {
    "psr-4": {
        "App\\": "src/"
    }
}
```

El archivo debe quedar así:

```
{
    "require": {
        "slim/slim": "4.*",
```



```
        "slim/psr7": "1.*"  
    },  
    "autoload": {  
        "psr-4": {  
            "App\\": "src/"  
        }  
    }  
}
```

Después ejecuta en terminal:

```
composer dump-autoload
```

Esto permitirá que las clases del espacio de nombres `App` se carguen automáticamente.

Paso 9 – Crear el modelo Incidente

Dentro de `src/Models`, crea el archivo `Incidente.php`:

```
<?php  
  
namespace App\Models;  
  
class Incidente  
{  
    public int $id;  
    public string $nombre;  
    public string $correo;  
    public string $rol;  
    public string $tipo;  
    public string $descripcion;  
    public ?string $evidencia;  
    public string $fecha_registro;  
  
    public function __construct(  
        int $id,  
        string $nombre,  
        string $correo,  
        string $rol,  
        string $tipo,  
        string $descripcion,  
        ?string $evidencia,  
        string $fecha_registro  
    ) {  
        $this->id = $id;
```



```
$this->nombre = $nombre;  
$this->correo = $correo;  
$this->rol = $rol;  
$this->tipo = $tipo;  
$this->descripcion = $descripcion;  
$this->evidencia = $evidencia;  
$this->fecha_registro = $fecha_registro;  
}  
}
```

El modelo representa la estructura lógica de un incidente dentro del sistema. Esta clase corresponde a la capa **Modelo**, ya que representa los datos con los que trabaja la aplicación.

Paso 10 – Crear la capa de acceso a datos

Dentro de `src/Data`, crea el archivo `DataContext.php`:

```
<?php  
  
namespace App\Data;  
  
use App\Models\Incidente;  
use mysqli;  
  
class DataContext  
{  
    private static mysqli $mysqli;  
    private array $settings;  
  
    public function __construct(array $settings)  
    {  
        $this->settings = $settings;  
        $this->conectar();  
    }  
  
    public function __destruct()  
    {  
        self::$mysqli->close();  
    }  
  
    protected function conectar()  
    {  
        self::$mysqli = new mysqli(  
            $this->settings['db']['host'],  
            $this->settings['db']['username'],
```



```
$this->settings['db']['password'],
$this->settings['db']['database']
);

if (self::$mysqli->connect_errno) {
    die('Error de conexión: ' . self::$mysqli->connect_error);
}
}

public function insertarIncidente(array $datos): bool
{
    $sql = "INSERT INTO incidente (nombre, correo, rol, tipo, descripcion, evidencia)
            VALUES (?, ?, ?, ?, ?, ?)";

    $stmt = self::$mysqli->prepare($sql);
    $stmt->bind_param(
        "ssssss",
        $datos['nombre'],
        $datos['correo'],
        $datos['rol'],
        $datos['tipo'],
        $datos['descripcion'],
        $datos['evidencia']
    );

    $resultado = $stmt->execute();
    $stmt->close();

    return $resultado;
}

public function obtenerIncidentes(): array
{
    $consulta = "SELECT * FROM incidente ORDER BY fecha_registro DESC";
    $resultado = self::$mysqli->query($consulta);

    $incidentes = [];

    while ($fila = $resultado->fetch_assoc()) {
        $incidentes[] = new Incidente(
            (int)$fila['id'],
            $fila['nombre'],
            $fila['correo'],
            $fila['rol'],
            $fila['tipo'],
            $fila['descripcion'],
            $fila['evidencia']
        );
    }
}
```



```
        $fila['descripcion'],  
        $fila['evidencia'],  
        $fila['fecha_registro']  
    );  
}  
  
    return $incidentes;  
}
```

Esta clase encapsula la conexión y las operaciones contra MySQL. Apoya a la capa Modelo, ya que permite almacenar y recuperar los datos que los modelos representan.

Paso 11 – Crear las rutas de la aplicación

Dentro de `app`, crea `routes.php`:

```
<?php  
  
use Slim\App;  
  
return function (App $app) {  
    $app->get('/', 'App\Controllers\HomeController:index');  
    $app->post('/incidente/guardar', 'App\Controllers\IncidenteController:guardar');  
    $app->get('/incidentes', 'App\Controllers\IncidenteController:listar');  
};
```

Las rutas indican qué controlador y qué método debe ejecutarse para cada URL.

Las rutas conectan el navegador con los controladores.

Paso 12 – Crear el controlador HomeController

Dentro de `src/Controllers`, crea `HomeController.php`:

```
<?php  
  
namespace App\Controllers;  
  
use Psr\Http\Message\ResponseInterface as Response;  
use Psr\Http\Message\ServerRequestInterface as Request;  
use Psr\Container\ContainerInterface;  
  
class HomeController  
{
```



```
public function __construct(private ContainerInterface $container)
{
}

public function index(Request $req, Response $res, $args)
{
    ob_start();
    require __DIR__ . '/../Views/Home/Index.php';
    $contenido = ob_get_clean();

    $res->getBody()->write($contenido);
    return $res;
}
}
```

Este controlador atiende la ruta principal / y muestra la vista del portal.
Corresponde a la capa **Controlador**, porque recibe la solicitud y decide qué vista debe renderizarse.

Paso 13 – Crear el controlador IncidenteController

Dentro de src/Controllers, crea IncidenteController.php:

```
<?php

namespace App\Controllers;

use App\Data\DataContext;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Container\ContainerInterface;

class IncidenteController
{
    private DataContext $db;

    public function __construct(private ContainerInterface $container)
    {
        $this->db = $container->get('db');
    }

    public function guardar(Request $req, Response $res, $args)
    {
        $datos = $req->getParsedBody();
        $archivos = $req->getUploadedFiles();
    }
}
```



```
$nombreArchivo = null;

if (isset($archivos['archivo-evidencia'])) {
    $archivo = $archivos['archivo-evidencia'];

    if ($archivo->getError() === UPLOAD_ERR_OK) {
        $nombreOriginal = $archivo->getClientFilename();
        $extension = pathinfo($nombreOriginal, PATHINFO_EXTENSION);
        $nombreArchivo = uniqid('evidencia_', true) . '.' . $extension;

        $rutaDestino = __DIR__ . '/../../public/uploads/' . $nombreArchivo;
        $archivo->moveTo($rutaDestino);
    }
}

$incidente = [
    'nombre' => $datos['nombre'] ?? '',
    'correo' => $datos['correo'] ?? '',
    'rol' => $datos['rol'] ?? '',
    'tipo' => $datos['tipo'] ?? '',
    'descripcion' => $datos['descripcion'] ?? '',
    'evidencia' => $nombreArchivo
];

$this->db->insertarIncidente($incidente);

return $res
    ->withHeader('Location', '/incidentes')
    ->withStatus(302);
}

public function listar(Request $req, Response $res, $args)
{
    $model = $this->db->obtenerIncidentes();

    ob_start();
    require __DIR__ . '/../Views/Incidente/Lista.php';
    $contenido = ob_get_clean();

    $res->getBody()->write($contenido);
    return $res;
}
}
```




El método `guardar`:

- Obtiene los datos enviados por el formulario.
- Obtiene el archivo subido.
- Genera un nombre único para evitar conflictos.
- Mueve el archivo a `public/uploads`.
- Guarda en base de datos el nombre del archivo y los datos del incidente.
- Redirige a la vista de incidentes registrados.

Este controlador coordina la lógica del proceso de guardado y consulta. Es la capa **Controlador**.

Paso 14 – Crear la vista principal del portal

Dentro de `src/Views/Home`, crea `Index.php`.

En esta vista reutiliza el diseño principal del **Portal Institucional del Centro de Cómputo FEI** trabajado en prácticas anteriores

Asegúrate de que el formulario quede así:

```
<form id="formulario-contacto" action="/incidente/guardar" method="post"
enctype="multipart/form-data" aria-label="Formulario de contacto">
```

Importante

El atributo:

```
enctype="multipart/form-data"
```

Es obligatorio para que la subida de archivos funcione.

El código del formulario debe quedar de la siguiente forma:

```
<form id="formulario-contacto" action="/incidente/guardar" method="post"
enctype="multipart/form-data" aria-label="Formulario de contacto">
  <fieldset>
    <legend>Datos del solicitante</legend>

    <p>
      <label for="nombre">Nombre completo</label><br>
      <input type="text" id="nombre" name="nombre" required>
    </p>

    <p>
      <label for="correo">Correo institucional</label><br>
```



```
<input type="email" id="correo" name="correo" required>
</p>

<p>
  <label for="rol">Rol</label><br>
  <select id="rol" name="rol" required>
    <option value="">Selecciona una opción</option>
    <option value="estudiante">Estudiante</option>
    <option value="docente">Docente</option>
    <option value="administrativo">Administrativo</option>
  </select>
</p>

<p>
  <label for="tipo">Tipo de solicitud</label><br>
  <select id="tipo" name="tipo" required>
    <option value="">Selecciona una opción</option>
    <option value="red">Red / Internet</option>
    <option value="cuenta">Cuenta / Acceso</option>
    <option value="software">Software</option>
    <option value="equipo">Equipo / Periféricos</option>
    <option value="otro">Otro</option>
  </select>
</p>

<p>
  <label for="descripcion">Descripción</label><br>
  <textarea id="descripcion" name="descripcion" rows="5" required></textarea>
</p>

<section id="evidencia" aria-labelledby="evidencia-titulo">
  <h3 id="evidencia-titulo">Carga de evidencia</h3>
  <p>Adjunta un archivo como evidencia del incidente.</p>

  <div id="zona-arrastre" tabindex="0">
    Arrastra aquí un archivo o haz clic para seleccionarlo.
  </div>

  <input type="file" id="archivo-evidencia" name="archivo-evidencia"
accept=".jpg,.jpeg,.png,.pdf,.doc,.docx">

  <p id="archivo-seleccionado">Ningún archivo seleccionado.</p>
</section>

<p>
```



```
<button type="submit">Enviar</button>
<button type="reset">Limpiar</button>
</p>
```

Agrega un enlace para consultar registros después de </form>:

```
<p><a href="/incidentes">Consultar incidentes registrados</a></p>
```

Esta es la capa **Vista**, ya que muestra la interfaz que usa el usuario.

Paso 15 – Crear la vista de consulta de incidentes

Dentro de src/Views/Incidente, crea Lista.php:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Incidentes registrados</title>
  <link rel="stylesheet" href="/css/estilos.css">
</head>
<body>
  <main id="contenido">
    <section>
      <h1>Incidentes registrados</h1>
      <p><a href="/">Volver al portal</a></p>

      <?php if (count($model) === 0): ?>
        <p>No existen incidentes registrados.</p>
      <?php else: ?>
        <?php foreach ($model as $incidente): ?>
          <article class="aviso">
            <h2><?= htmlspecialchars($incidente->tipo) ?></h2>
            <p><strong>Nombre:</strong> <?= htmlspecialchars($incidente->nombre)
?></p>
            <p><strong>Correo:</strong> <?= htmlspecialchars($incidente->correo)
?></p>
            <p><strong>Rol:</strong> <?= htmlspecialchars($incidente->rol)
?></p>
            <p><strong>Descripción:</strong> <?= htmlspecialchars($incidente->descripcion) ?></p>
            <p><strong>Fecha:</strong> <?= htmlspecialchars($incidente->fecha_registro) ?></p>
```



```
<?php if (!empty($incidente->evidencia)): ?>
    <p>
        <strong>Evidencia:</strong>
        <a href="/uploads/<?= htmlspecialchars($incidente-
>evidencia) ?>" target="_blank">
            Ver archivo
        </a>
    </p>
<?php endif; ?>
</article>
<?php endforeach; ?>
<?php endif; ?>
</section>
</main>
</body>
</html>
```

Esta vista recorre el arreglo de incidentes recibido desde el controlador y los muestra uno por uno. Corresponde a la capa **Vista**.

Paso 16 – Configurar el archivo principal `index.php`

En la carpeta **public/** crea `index.php`:

```
<?php

use DI\Container;
use Psr\Container\ContainerInterface;
use Slim\Factory\AppFactory;

require __DIR__ . '/../vendor/autoload.php';

// Crear contenedor
$container = new Container();

// Registrar configuración
$container->set('settings', function () {
    $settings = require __DIR__ . '/../app/settings.php';
    return new \App\Settings\Settings($settings);
});

// Registrar conexión a base de datos
```



```
$container->set('db', function (ContainerInterface $container) {  
    return new \App\Data\DataContext(  
        $container->get('settings')->get()  
    );  
});  
  
// Configurar Slim con el contenedor  
AppFactory::setContainer($container);  
$app = AppFactory::create();  
  
// Cargar rutas  
$routes = require __DIR__ . '/../app/routes.php';  
$routes($app);  
  
// Middlewares importantes  
$app->addBodyParsingMiddleware();  
$app->addRoutingMiddleware();  
$app->addErrorMiddleware(true, true, true);  
  
// Ejecutar app  
$app->run();
```

Este archivo:

- Carga Composer
- Crea el contenedor de dependencias
- Registra servicios
- Carga rutas
- Habilita el análisis del cuerpo de la petición
- Ejecuta la aplicación

Este archivo es el punto de entrada principal de la aplicación. No es una capa MVC por sí mismo, pero conecta toda la arquitectura.

Paso 17 – Copiar recursos estáticos al directorio público si aún no lo has hecho

Copia tu hoja de estilos a:

public/css/estilos.css

Copia las imágenes a:

public/img

Copia los datos a:

public/datos/

Crea la carpeta:

public/uploads



Asegúrate de que `public/uploads` tenga permisos de escritura.

Paso 18 – Ejecutar la aplicación

En la terminal, desde la raíz del proyecto, ejecuta:

```
php -S localhost:8080 -t public
```

`php -S` inicia un servidor web local para probar aplicaciones PHP desde el navegador.

Abre tu navegador en:

<http://localhost:8080>

Si ves algo como lo siguiente:

The screenshot shows the 'Portal Institucional del Centro de Cómputo' website. At the top, there is a header with the logo of the Facultad de Estadística e Informática, the name 'Universidad Veracruzana', and the faculty name. Below this is a navigation menu with links: 'Avisos', 'Servicios', 'Horarios', 'Indicadores', and 'Contacto'. The main content area has a section titled 'Estado del portal' with a green banner stating 'Se cargaron 3 avisos correctamente.' Below this is the 'Portal Institucional del Centro de Cómputo' title, followed by a paragraph of information and links for 'Ver servicios' and 'Reportar incidente'. There is a 'Galería del Centro de Cómputo' section with a sub-header 'Imagen del Centro de Cómputo' and a description 'Vista general del laboratorio.' with 'Anterior' and 'Siguiente' buttons. The 'Avisos y comunicados' section lists three items: 'Mantenimiento programado de red' (dated 10 de junio de 2026), 'Actualización de software en laboratorios' (dated 12 de junio de 2026), and 'Convocatoria para préstamo de equipo'.

Paso 19 – Elimina los archivos innecesarios de la práctica anterior

Elimina los archivos `index.html`, la carpeta `css` y la carpeta `img` de la raíz del proyecto

Paso 20 – Ajusta las rutas de la hoja de estilos, las imágenes y el script

Los archivos estáticos deben estar en:



public/css
public/img

La vista Index.php en:

src/Views/Home/Index.php

El archivo index.php en:

public/index.php

En la vista debes referenciarlos con rutas **desde la raíz pública del sitio**, no con rutas relativas.

Aunque la vista esté guardada físicamente en:
src/Views/Home/Index.php

El navegador **no conoce esa ruta interna del proyecto**.

El navegador solo ve la parte pública del sitio, es decir, lo que expones desde public/.
Por eso:

public/css/estilos.css se accede como /css/estilos.css

public/img/logo-fei.jpeg se accede como /img/logo-fei.jpeg

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Portal institucional: avisos, servicios, horarios,
indicadores y contacto.">
  <title>Portal Institucional | Centro de Cómputo</title>
  <link rel="stylesheet" href="/css/estilos.css">
  <link rel="icon" href="/img/favicon_fei.ico" type="image/x-icon">
  <script src="/js/script.js" defer></script>
</head>
```

Paso 21 – Elimina funciones de script.js que interfieren con el envío de datos del formulario

De script.js, elimina el código correspondiente a //Convertir datos del formulario a JSON:

```
//Convertir datos del formulario a JSON

document.getElementById("formulario-contacto").addEventListener("submit", function (event) {
  event.preventDefault();

  const solicitud = {
    nombre: document.getElementById("nombre").value,
    correo: document.getElementById("correo").value,
```



```
rol: document.getElementById("rol").value,  
tipo: document.getElementById("tipo").value,  
descripcion: document.getElementById("descripcion").value  
};  
  
/*Convierte el objeto javascript "solicitud" en una cadena JSON  
estableciendo el parámetro replacer=null y el parámetro space=2*/  
const solicitudJSON = JSON.stringify(solicitud, null, 2);  
  
document.getElementById("salida-json").textContent = solicitudJSON;  
});
```

Elimina la función: //Validación de formulario

Y ambas sustitúyelas por:

```
//Envío de formulario  
const formularioContacto = document.getElementById("formulario-contacto");  
  
if (formularioContacto) {  
    formularioContacto.addEventListener("submit", function (event) {  
  
        const solicitud = {  
            nombre: document.getElementById("nombre").value,  
            correo: document.getElementById("correo").value,  
            rol: document.getElementById("rol").value,  
            tipo: document.getElementById("tipo").value,  
            descripcion: document.getElementById("descripcion").value  
        };  
  
        const solicitudJSON = JSON.stringify(solicitud, null, 2);  
  
        const salidaJSON = document.getElementById("salida-json");  
        if (salidaJSON) {  
            salidaJSON.textContent = solicitudJSON;  
        }  
  
        if (campoNombre.value.trim() === "") {  
            event.preventDefault();  
            mensajeFormulario.textContent = "El nombre completo es obligatorio.";  
            mensajeFormulario.style.color = "red";  
            campoNombre.focus();  
            return;  
        }  
    }  
}
```




```
if (!campoCorreo.value.includes("@")) {
    event.preventDefault();
    mensajeFormulario.textContent = "El correo institucional no es válido.";
    mensajeFormulario.style.color = "red";
    campoCorreo.focus();
    return;
}

if (campoRol.value === "") {
    event.preventDefault();
    mensajeFormulario.textContent = "Debes seleccionar un rol.";
    mensajeFormulario.style.color = "red";
    campoRol.focus();
    return;
}

if (campoTipo.value === "") {
    event.preventDefault();
    mensajeFormulario.textContent = "Debes seleccionar un tipo de solicitud.";
    mensajeFormulario.style.color = "red";
    campoTipo.focus();
    return;
}

if (campoDescripcion.value.trim().length < 10) {
    event.preventDefault();
    mensajeFormulario.textContent = "La descripción debe contener al menos 10
caracteres.";
    mensajeFormulario.style.color = "red";
    campoDescripcion.focus();
    return;
}

if (!archivoActual) {
    event.preventDefault();
    mensajeFormulario.textContent = "Debes adjuntar un archivo de evidencia.";
    mensajeFormulario.style.color = "red";
    zonaArrastre.focus();
    return;
}

mensajeFormulario.textContent = "Formulario validado correctamente. Enviando datos...";
mensajeFormulario.style.color = "green";
});
```



```
}
```

Finalmente la funcionalidad drag and drop del archivo de evidencia la cambiaremos por un bloque de código que sí envíe el archivo al backend.

Identifica el siguiente bloque de código:

```
/* Soltar archivo en la zona */
zonaArrastre.addEventListener("drop", (event) => {
  event.preventDefault();
  zonaArrastre.classList.remove("activa");

  try {
    const archivos = event.dataTransfer.files;

    if (archivos.length === 0) {
      throw new Error("No se arrastró ningún archivo.");
    }

    archivoActual = archivos[0];
    mostrarArchivo(archivoActual);
  } catch (error) {
    archivoActual = null;
    archivoSeleccionado.textContent = "Error: " + error.message;
    archivoSeleccionado.style.color = "red";
  }
});
```

Sustitúyelo por:

```
/* Soltar archivo en la zona */
zonaArrastre.addEventListener("drop", (event) => {
  event.preventDefault();
  zonaArrastre.classList.remove("activa");

  try {
    const archivos = event.dataTransfer.files;

    if (archivos.length === 0) {
      throw new Error("No se arrastró ningún archivo.");
    }

    archivoActual = archivos[0];
```



```
// Asignar el archivo al input para que el formulario lo envíe realmente al backend
const dataTransfer = new DataTransfer();
dataTransfer.items.add(archivoActual);
inputArchivo.files = dataTransfer.files;

mostrarArchivo(archivoActual);
} catch (error) {
    archivoActual = null;
    archivoSeleccionado.textContent = "Error: " + error.message;
    archivoSeleccionado.style.color = "red";
}
});
```

Paso 22 – Probar el funcionamiento completo

Verifica lo siguiente:

- La vista principal del portal carga correctamente sin errores en consola.
- El formulario aparece dentro del portal con su estructura visual.
- El formulario permite capturar nombre, correo, rol, tipo y descripción.
- Es posible seleccionar un archivo de evidencia.
- Al enviar el formulario, los datos se almacenan en MySQL.
- El archivo de evidencia se guarda en `public/uploads`.
- La aplicación redirige a la vista `/incidentes` después de enviar un incidente.
- La lista de incidentes muestra los registros capturados.
- Si existe archivo de evidencia, aparece el enlace para abrirlo.
- El enlace para volver al portal funciona correctamente.

Paso 20 – Reflexionar sobre la arquitectura MVC

Una vez que la aplicación funcione, identifica la relación entre sus partes:

Modelo (**Incidente.php**)

Representa la información de un incidente.

Vista (**Index.php**, **Lista.php**)

Presenta la información al usuario.

Controladores (**HomeController.php**, **IncidenteController.php**)

Atienden solicitudes, coordinan procesos y eligen qué vista mostrar.

Acceso a datos (**DataContext.php**)

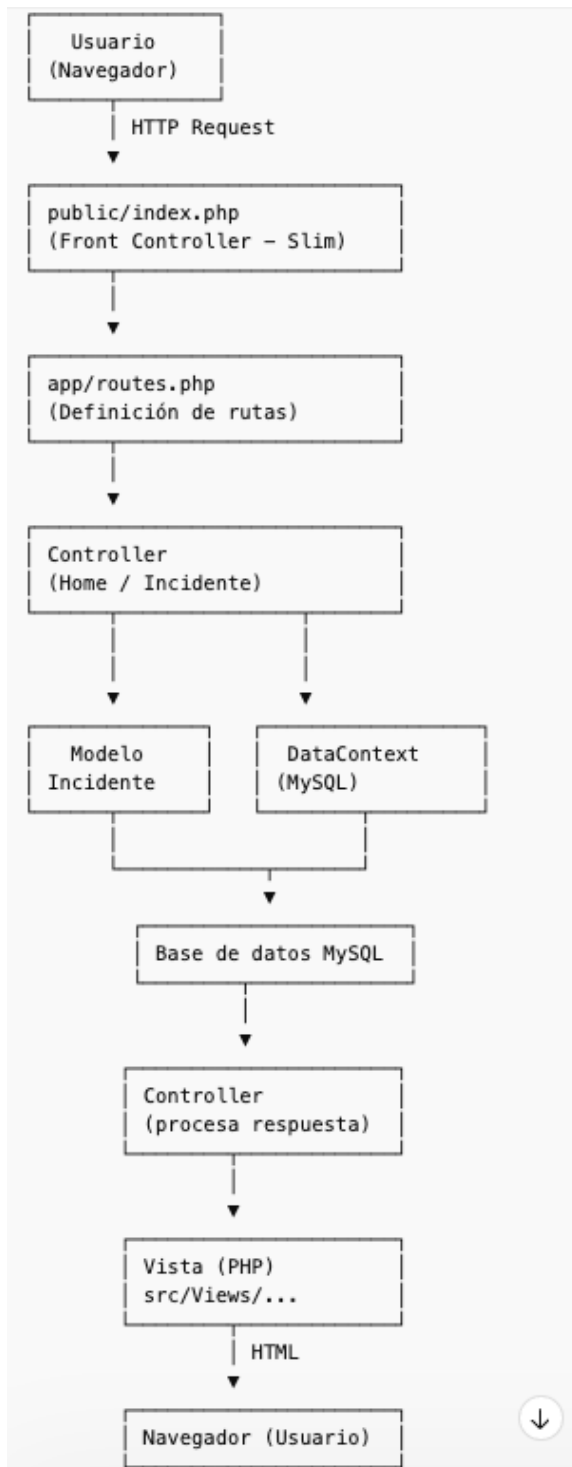
Realiza operaciones con MySQL.

Rutas (**routes.php**)

Relacionan las URL con los controladores.



Por lo tanto el flujo MVC del proyecto es el siguiente:





Esta reflexión es importante porque te ayuda a comprender que una aplicación web bien estructurada distribuye responsabilidades entre capas, en lugar de concentrarlo todo en un solo archivo PHP.

RESULTADOS ENTREGABLES

1. Un archivo .zip con la carpeta completa del proyecto.
2. El proyecto debe incluir correctamente organizados los siguientes elementos:
 - index.php
 - composer.json
 - bd.sql
 - carpeta app
 - carpeta src
 - carpeta public
1. La aplicación debe funcionar correctamente en navegador.
2. La base de datos debe almacenar los reportes capturados.
3. Debe existir una vista para consultar los incidentes registrados.
4. Debe ser posible subir un archivo de evidencia real y consultarlo desde la lista de incidentes.
5. El código debe mantener estructura ordenada, indentación adecuada y comentarios en las secciones principales.
6. Entregar capturas de pantalla que evidencien:
 - La estructura del proyecto MVC
 - La ejecución de la aplicación sin errores en consola
 - La base de datos con registros
 - La vista de incidentes registrados